



Proyecto (TB)

Matemática Computacional



Tipo de participación	Grupal (5 estudiantes como máximo)
Fecha de inicio del TP	Semana 1
Fecha de los avances	- Al finalizar de la semana 6, los estudiantes deberán entregar la introducción, los objetivos, la metodología (algoritmos o métodos aplicados), un avance del 50 % del código fuente y del programa ejecutable (librerías utilizadas, descripción, búsqueda de soluciones óptimas y visualización de resultados), así como la bibliografía del informe. Esta revisión tendrá un puntaje máximo de 8 puntos. - Al finalizar la semana 11 presentan el 100 % del código fuente y del programa ejecutable (100% del código del proyecto, diagrama de flujo, resultados y conclusiones. La sustentación tiene un puntaje máximo de 12 puntos
Fecha de sustentación	Semana 12
Entregable	Al finalizar la semana 11 se presentarán todos los archivos (informe, presentación, código fuente y programa ejecutable).
Insumos	Matemática Computacional
Descripción	Cada grupo debe desarrollar un programa, incluyendo el código fuente y el ejecutable en cualquier lenguaje de programación. Se considerará únicamente el aporte del grupo, excluyendo la copia de bibliotecas. Sin embargo, las librerías pueden utilizarse para optimizar el desarrollo, ya que facilitan la reutilización de código y el acceso a funcionalidades adicionales, pero no se incluirán en el porcentaje de avance. 1. Componentes conexas de un grafo Se deberá desarrollar una aplicación con una interfaz amigable e interactiva que permita al usuario participar activamente en cada etapa de la ejecución del algoritmo de componentes conexas. Inicialmente, el programa solicitará un número entero n con $n \in [4, 12]$, correspondiente a la cantidad de nodos del grafo, y posteriormente permitirá elegir entre generar el grafo de forma manual, definiendo las conexiones entre los vértices, o de manera aleatoria. Una vez construido el grafo, el sistema mostrará la matriz de adyacencia resultante y ejecutará el algoritmo de identificación de componentes conexas, presentando detalladamente cada paso del proceso. Durante la ejecución, el usuario podrá avanzar de forma interactiva por las distintas etapas del algoritmo para observar cómo se analizan los vértices y cómo se forman las componentes. Finalmente, el programa mostrará la solución completa, indicando el número total de componentes conexas encontradas y la lista de nodos que conforman cada una de ellas, junto con la representación gráfica del grafo y el resultado obtenido para facilitar su interpretación y comprensión. 2. Problema del camino mínimo Se deberá desarrollar una aplicación con una interfaz amigable e interactiva que permita al usuario participar activamente en cada etapa de la ejecución del algoritmo de Dijkstra. Inicialmente, el programa solicitará un número entero n con $n \in [7, 16]$, correspondiente a la cantidad de nodos del grafo, y permitirá elegir entre generar el grafo de forma manual o aleatoria. En caso de optar por la generación manual, el usuario podrá definir las conexiones entre los vértices e ingresar los pesos asociados a cada arista. El grafo generado deberá ser dirigido y no contener ciclos. Una vez construido, el sistema mostrará su representación gráfica etiquetada y solicitará la selección de un vértice origen y un vértice destino. A continuación, el programa ejecutará el algoritmo de Dijkstra mostrando detalladamente cada paso del procedimiento, incluyendo el etiquetado de los vértices, la actualización de las distancias mínimas, la selección de nodos visitados y las

decisiones tomadas en cada iteración. Durante este proceso, el usuario podrá avanzar de forma interactiva para observar y comprender el funcionamiento del algoritmo paso a paso. Finalmente, el sistema presentará la solución completa indicando la distancia mínima encontrada entre el vértice origen y el vértice destino, la cantidad de caminos mínimos existentes y la secuencia de vértices que conforma cada uno de ellos, resaltándolos visualmente sobre el grafo para facilitar su interpretación y análisis.

3. Problema del flujo máximo

Se deberá desarrollar una aplicación con una interfaz amigable e interactiva que permita al usuario participar activamente en cada etapa de la ejecución del algoritmo de Ford-Fulkerson. Inicialmente, el programa solicitará un número entero n con $n \in [7, 16]$, correspondiente a la cantidad de nodos del grafo dirigido, y permitirá elegir entre generar el grafo de forma manual o aleatoria. En el caso de la creación manual, el usuario podrá definir las aristas e ingresar la capacidad asociada a cada una de ellas. El sistema deberá verificar que el grafo no contenga ciclos; en caso de detectarse alguno, deberá informar al usuario y solicitar la corrección de la estructura antes de continuar con la ejecución del algoritmo. Una vez construido el grafo, este se mostrará de manera gráfica y etiquetada, permitiendo al usuario seleccionar un vértice fuente y un vértice sumidero. Adicionalmente, el programa deberá contemplar escenarios en los que sea necesario incorporar un origen ficticio y/o un destino ficticio para modelar problemas con múltiples fuentes o múltiples sumideros.

A continuación, el sistema ejecutará el algoritmo de Ford-Fulkerson mostrando detalladamente cada etapa del proceso. El usuario podrá avanzar de manera interactiva paso a paso para observar la identificación de caminos aumentantes, la determinación de capacidades residuales, la actualización de los flujos en las aristas y la construcción de la red residual en cada iteración. Durante el desarrollo del algoritmo se deberá visualizar claramente cómo evoluciona el flujo total de la red y cómo se modifican las capacidades disponibles. Finalmente, el programa presentará la solución completa indicando el valor del flujo máximo obtenido entre la fuente y el sumidero, la asignación de flujo en cada arista y el detalle de los caminos aumentantes utilizados durante el procedimiento. Asimismo, deberá identificar y resaltar sobre el grafo un corte mínimo que certifique la maximalidad del flujo encontrado, permitiendo al usuario comprender la relación entre el flujo máximo y el corte mínimo mediante una representación visual clara y de fácil interpretación.

4. Problema del agente viajero

Se deberá desarrollar una aplicación con una interfaz amigable e interactiva que permita al usuario participar activamente en cada etapa de la ejecución del Problema del Agente Viajero mediante el algoritmo de fuerza bruta. Inicialmente, el programa solicitará un número entero n con $n \in [5, 10]$, correspondiente a la cantidad de nodos del grafo no dirigido, y permitirá al usuario elegir entre generarlo de forma manual o aleatoria. En caso de optar por la generación manual, el usuario deberá definir las aristas e ingresar los pesos asociados a cada una de ellas, los cuales representarán distancias, costos o tiempos de recorrido entre los nodos. El sistema deberá verificar que la estructura construida permita la existencia de ciclos hamiltonianos; en caso contrario, deberá informar al usuario cuáles son las aristas que faltan incorporar para completar al menos un ciclo hamiltoniano válido. Una vez construido el grafo ponderado, el programa mostrará su representación gráfica etiquetada y procederá a identificar, de manera detallada y paso a paso, todos los ciclos hamiltonianos existentes. Durante este proceso, el usuario podrá interactuar con la aplicación avanzando iterativamente por cada etapa del algoritmo y observando cómo se generan y evalúan las

	diferentes rutas posibles. Posteriormente, el sistema construirá la matriz de costos correspondiente y aplicará el algoritmo de fuerza bruta para calcular el costo total asociado a cada ciclo hamiltoniano, comparando todas las soluciones obtenidas. Finalmente, el programa presentará la solución completa indicando el número total de ciclos hamiltonianos encontrados, el costo total de cada uno de ellos y el ciclo hamiltoniano óptimo que minimiza la distancia, el costo o el tiempo total del recorrido. Adicionalmente, dicho ciclo deberá resaltarse visualmente sobre el grafo para facilitar su análisis, permitiendo al usuario comprender claramente cómo se obtuvo la solución óptima del Problema del Agente Viajero.
Condiciones	• Cada grupo hará sólo una de estas tareas (elegida por sorteo). • Cada revisión se calificará en base a una rúbrica. • Si un grupo decide expulsar a un integrante, deben tener evidencias de que no ha aportado en el proyecto. Además, el expulsado ya no puede integrar otro grupo. • Un integrante puede cambiar de grupo si los demás compañeros de grupo no están cumpliendo con los avances y aún faltan más de dos semanas para la primera revisión.